Universiteit Antwerpen

Introduction to Databases
Lecture 8:
3rd Normal Form

Floris Geerts

Universiteit Antwerpen

Last lecture:

- FDs can flag problems in our relation:
  - Non-trivial X→Y with X not a superkey
- Boyce-Codd Normal Form indicates a well-designed relation not having this problem
- If a relation does not satisfy BCNF we can solve this by decomposing
  - Decomposition is lossless
  - Decomposition is in BCNF

However: decomposition is not always *dependency-preserving*!

This means that checking constraints can be *very expensive*

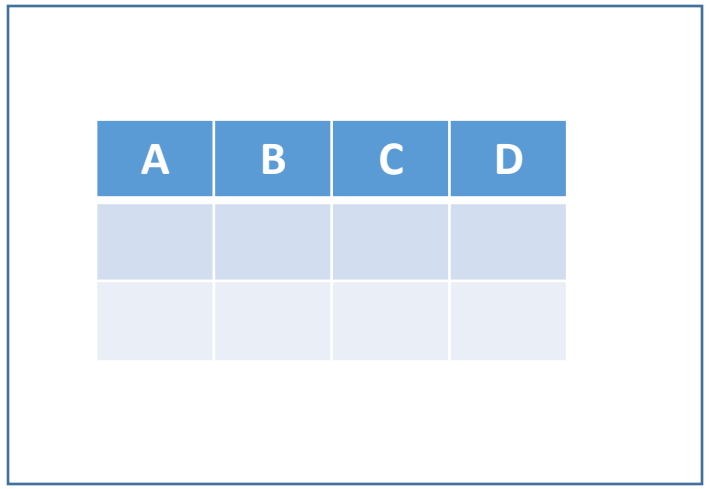
Desirable properties of decompositions

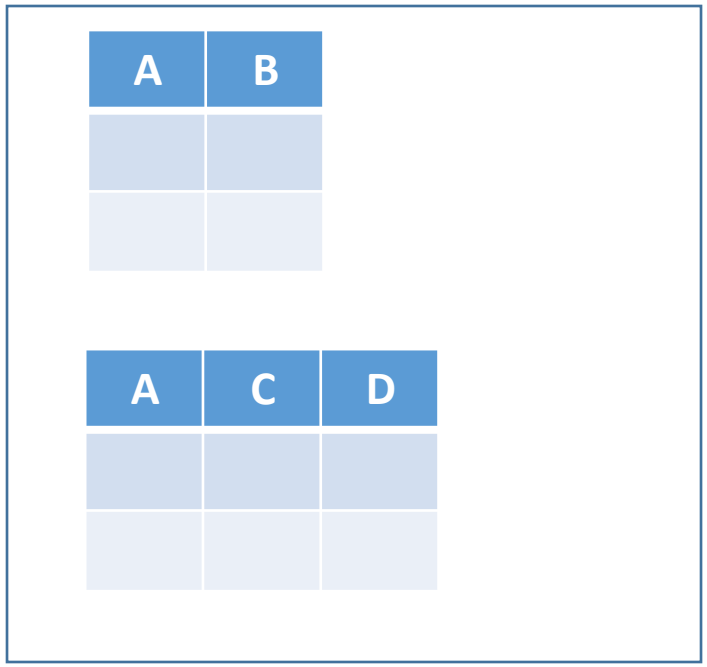
- **Dependency-preserving**: A decomposition $R = R_1 \bowtie R_2 \bowtie \ldots \bowtie R_k$ under a set of functional dependencies F is dependency-preserving, if and only if $F^+ = (F_1 \cup F_2 \cup \ldots \cup F_k)^+$,
  where $F_i = \{ X \rightarrow Y \in F^+ \mid X, Y \subseteq$ attributes of $R_i \}$ for i=1…k

- Hence no expensive triggers required to enforce consistency

Dependency-Preserving: Example

- Decomposing R(A,B,C,D) under FD = { A→CD, B→D, BC→A }
  - $R_1 = \pi_{AB}R$ and $R_2 = \pi_{ACD}R$ is not dependency-preserving:
    - B→D is not in $(F_1 \cup F_2)^+$
    - BC→A is not in $(F_1 \cup F_2)^+$





Dependency-Preserving: Example

- Decomposing R(A,B,C,D) under FD = { A→CD, B→D, BC→A }
  - $R_1 = \pi_{AB}R$ and $R_2 = \pi_{ACD}R$ is not dependency-preserving:
    - B→D is not in $(F_1 \cup F_2)^+$
    - BC→A is not in $(F_1 \cup F_2)^+$

insert values (1,2,3,4) into R

A	B	C	D
1	2	3	4

insert values (1,2) into R1

insert values (1,3,4) into R2

A	B
1	2

A	C	D
1	3	4

Universiteit Antwerpen

Dependency-Preserving: Example

- Decomposing R(A,B,C,D) under FD = { A→CD, B→D, BC→A }
  - $R_1 = \pi_{AB}R$ and $R_2 = \pi_{ACD}R$ is not dependency-preserving:
    - B→D is not in $(F_1 \cup F_2)^+$
    - BC→A is not in $(F_1 \cup F_2)^+$

insert values (5,2) into R1

insert values (5,6,7) into R2

insert values (5,2,6,7) into R

A	B	C	D
1	2	3	4
~~5~~	~~2~~	~~6~~	~~7~~

B→D violated

A	B
1	2
5	2

A	C	D
1	3	4
5	6	7

Universiteit Antwerpen

Dependency-Preserving: Example

- Decomposing R(A,B,C,D) under FD = { A→CD, B→D, BC→A }
  - We cannot invalidate R1 without looking at R2:

A	B	C	D
1	2	3	4
5	2	6	7

A	B
1	2
5	2

A	C	D
1	3	4
5	6	7

Dependency-Preserving: Example

- Decomposing R(A,B,C,D) under FD = { A→CD, B→D, BC→A }
  - We cannot invalidate R1 without looking at R2:

A	B	C	D
1	2	3	4
5	2	6	4

A	B
1	2
5	2

A	C	D
1	3	4
5	6	7

Dependency-Preserving: Example

- Decomposing R(A,B,C,D) under FD = { A→CD, B→D, BC→A }
  - We cannot invalidate R2 without looking at R1:

A	B	C	D
1	2	3	4
5	2	6	7

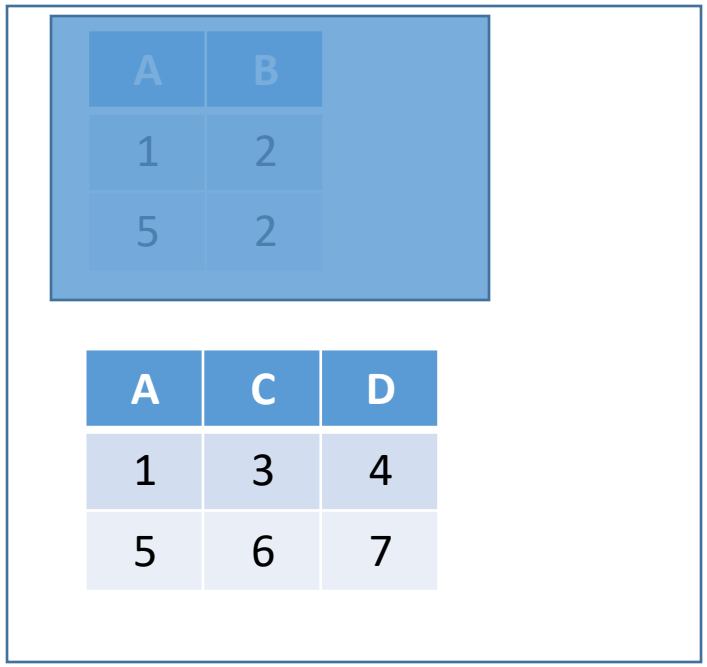
A	B
1	2
5	2

A	C	D
1	3	4
5	6	7

Dependency-Preserving: Example

- Decomposing R(A,B,C,D) under FD = { A→CD, B→D, BC→A }
  - We cannot invalidate R2 without looking at R1:

A	B	C	D
1	2	3	4
5	8	6	7



Example: Non-dependency preserving

Consider a relation

Bookings(movie, theater, city)

"*movie* plays in *theater* located in *city*"

- Assume non-trivial FD's:
  - theater → city
  - movie,city → theater
- Then, the keys are: {movie, city} and {theater, movie}
- **This relation is not in BCNF!**
  - theater→city is non-trivial and theater is not a superkey

Example: Non-dependency preserving

- Only way to losslessly split it:
  - R1(theater, city)
  - R2(theater, movie)
- R1 and R2 are in BCNF
- **However, FD movie,city → theatre is not preserved!**

Example: Non-dependency preserving

- Only way to losslessly split it:
  - R1(theater, city)
  - R2(theater, movie)
- R1 and R2 are in BCNF
- **However, FD movie,city → theatre is not preserved!**

theater	city	movie
Kinepolis	Antwerpen	Tenet
UGC	Antwerpen	Sint, the movie

theater	city	movie
Kinepolis	Antwerpen	Tenet
UGC	Mechelen	Tenet

These two relations both satisfy all dependencies
They are incompatible, but both represent a possible world,
fully consistent with our constraints

Example: Non-dependency preserving

- Only way to losslessly split it:
  - R1(theater, city)
  - R2(theater, movie)
- R1 and R2 are in BCNF
- **However, FD movie,city → theatre is not preserved!**

theater	city
Kinepolis	Antwerpen
UGC	Antwerpen

theater	movie
Kinepolis	Tenet
UGC	Tenet

So, looking at R1 and R2 in isolation, there is no way that we can reject any of the inserts/updates that lead to these instances

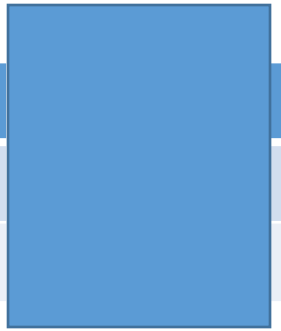
Example: Non-dependency preserving

- Only way to losslessly split it:
  - R1(theater, city)
  - R2(theater, movie)
- R1 and R2 are in BCNF
- **However, FD movie,city → theatre is not preserved!**

theater	city
Kinepolis	Antwerpen
UGC	Antwerpen

⋈

theater	movie
Kinepolis	Tenet
UGC	Tenet



theater	city	movie
Kinepolis	Antwerpen	Tenet
UGC	Antwerpen	Tenet

Together, however, they represent an illegal situation that violates city,movie→theater!

Universiteit Antwerpen

Example: Non-dependency preserving

- Only way to losslessly split it:
  - R1(theater, city)
  - R2(theater, movie)
- R1 and R2 are in BCNF
- **However, FD movie,city → theatre is not preserved!**
  - To avoid such inconsistencies, we hence need triggers or assertions that check inserts/updates in R1 and R2. These triggers need to join both tables in order to check movie,city → theatre !
  - This is expensive and hence undesirable
- Therefore, we may choose to settle for “a little redundancy” in order to preserve all dependencies

3rd Normal Form (3NF)

- Dependency-preservation is a desirable property as it avoids costly triggers/assertions
- The third normal form (3NF) represents the farthest we can go while preserving dependencies:

A relation schema R is in Third Normal Form (3NF) with respect to a set of dependencies F, if for every non-trivial FD X → A in F it holds that X is a superkey of R *or A is an attribute of a key of R*.

Example: Relation in 3NF, not in BCNF

Recall the relation Bookings(movie, theater, city)

theater → city, movie,city → theater

- The keys are: {movie, city} and {theater, movie}
- **This relation is not in BCNF!**
  - theater→city is non-trivial and theater is not a superkey
- **Relation is in 3NF**
  - City is part of the key {movie,city}

3rd Normal Form (3NF)

Given a relation, we can always find *at least one* decomposition that is at the same time:

- Lossless
- Dependency-preserving
- 3NF

This does **not** mean that *every* decomposition in 3NF is dependency-preserving!

We will give an algorithm that always produces a lossless, dependency-preserving, 3NF decomposition

How to Test for Dependency-Preservation?

Consider a decomposition $R = R1 \cup \ldots \cup Rk$, and a set of FDs F

We need to check if $F^+ = (F_1 \cup F_2 \cup \ldots \cup F_k)^+$

Recall $F_i = \{ X \rightarrow A \in F^+ \mid X \cup \{A\} \subseteq R_i \} \subseteq F^+$

Hence $(F_1 \cup F_2 \cup \ldots \cup F_k)^+ \subseteq F^{++} = F^+$

Therefore:

- it suffices to show that $F^+ \subseteq (F_1 \cup F_2 \cup \ldots \cup F_k)^+$
- it suffices to show that $F \subseteq (F_1 \cup F_2 \cup \ldots \cup F_k)^+$
  Indeed, if all FDs from F can be derived, also all FDs derivable from F can be derived!

How to Test for Dependency-Preservation?

Consider a decomposition $R = R1 \cup \ldots \cup Rk$, and a set of FDs F

The decomposition is dependency-preserving if and only if

$$F \subseteq (F_1 \cup F_2 \cup \ldots \cup F_k)^+$$

Algorithm:

- For every FD $X \rightarrow A \in F$:
  - Check if $A \in X^{+(F1 \cup F2 \cup \ldots \cup Fk)}$

How to Test for Dependency-Preservation?

Consider a decomposition $R = R1 \cup \ldots \cup Rk$, and a set of FDs F

The decomposition is dependency-preserving if and only if

$$F \subseteq (F_1 \cup F_2 \cup \ldots \cup F_k)^+$$

Algorithm:

- For every FD $X \rightarrow A \in F$:

  // Check if $A \in X^{+(F1 \cup F2 \cup \ldots \cup Fk)}$

  while X updating:

    for i = 1…k, $Y \rightarrow B \in F_i$:

      if $Y \subseteq X$: $X := X \cup \{B\}$

$Y \rightarrow B \in F_i$ iff $B \in Y^{+(F)}$ and $Y \cup \{B\} \subseteq R_i$

Hence, we can add to X, every element of $(X \cap R_i)^{+(F)} \cap R_i$

How to Test for Dependency-Preservation?

Consider a decomposition $R = R1 \cup \ldots \cup Rk$, and a set of FDs F

The decomposition is dependency-preserving if and only if

$$F \subseteq (F_1 \cup F_2 \cup \ldots \cup F_k)^+$$

Algorithm:

- For every FD $X \rightarrow A \in F$:
  - while X updating:
    - for i = 1…k:
      - $X := X \cup ((X \cap R_i)^{+(F)} \cap R_i)$
  - if $A \notin X$: return False
- Return True

Example: Test for Dependency-Preservation

R(T,C,M) decomposed into R1(T,C) and R2(T,M)

F = { T→C, MC→T }

T→C is obviously preserved as it is in F_1

For MC→T, we check if T is in $MC^{+(F1 \cup F2)}$

- F_1 allows to add $(MC \cap TC)^{+(F)} \cap TC = C^{+(F)} \cap TC = C$
- F_2 allows to add $(MC \cap TM)^{+(F)} \cap TM = M^{+(F)} \cap TM = M$

Hence, $MC^{+(F1 \cup F2)} = MC$ and thus **MC→T is not preserved**

Example 2: Test for Dependency-Preservation

R(A,B,C,D) decomposed into R_1(A,C), R_2(A,D), R_3(B,D)

F={A→D, AB→C, D→B}

A→D and D→B trivially preserved

AB→C ? We need to check if C is in $AB^{+(F1 \cup F2 \cup F3)}$

- F_1 allows to add $(AB \cap AC)^{+(F)} \cap AC = A^{+(F)} \cap AC = ABCD \cap AC = AC$
  Hence, X becomes ABC
- *F_2 allows to add $(ABC \cap AD)^{+(F)} \cap AD = A^{+(F)} \cap AD = ABCD \cap AD = AD$*
  Hence, X becomes ABCD

C is in $AB^{+(F1 \cup F2 \cup F3)}$, **thus AB→C is preserved!**

Universiteit Antwerpen

Example 3: Test for Dependency-Preservation

R(A,B,C) decomposed into $R_1(A,B)$, $R_2(A,C)$

F={A→B, B→C, C→A}

A→B and C→A trivially preserved

B→C ? We need to check if C is in $B^{+(F1 \cup F2)}$

- F_1 allows to add $(B \cap AB)^{+(F)} \cap AB = B^{+(F)} \cap AB = ABC \cap AB = AB$
  Hence, X becomes AB
- F_2 allows to add $(AB \cap AC)^{+(F)} \cap AC = A^{+(F)} \cap AC = ABC \cap AC = AC$
  Hence, X becomes ABC

C is in $B^{+(F1 \cup F2 \cup F3)}$, **thus B→C is preserved!**

Algorithm for Decomposing in 3NF

- In the decomposition we want to preserve every dependency
  - First minimize our set of dependencies
    - Split up all right-hand sides: A→BC is equivalent to A→B and A→C
    - Remove superfluous attributes

      AB→C, A→B is equivalent to A→C, A→B
    - Remove superfluous dependencies

      A→B and B→C imply A→C, so if we preserve A→B and B→C then also A→C is preserved
  - Make sure at least one relation supports every remaining dependency
  - Make sure the decomposition is lossless

1. Compute Minimal Basis

- A minimal basis B for a set of FDs F is a set of dependencies such that:
  - $B^+=F^+$
  - B is minimal in the following sense:
    - All FDs in B are of the form X→A where A is a single attribute
    - For every X→A in B, it holds that:
      - $(B \setminus \{X \rightarrow A\})^+ \neq B^+$
    - For every XY→A in B, it holds that:
      - $(B \setminus \{XY \rightarrow A\} \cup \{X \rightarrow A\})^+ \neq B^+$

    In other words: we cannot remove any dependency or attribute from B without changing B^+
- How to compute? Short answer:
  - *Test for every attribute, dependency if removing it changes B^+; if not remove. Continue until no more changes*

1. Computing Minimal Basis

Test for every dependency, attribute if removing it changes B^+

- We can replace $XA \rightarrow Y$ with $X \rightarrow Y$ if:
  - $B^+ = (B \setminus \{XA \rightarrow Y\}) \cup \{X \rightarrow Y\})^+$
  - Since $X \rightarrow Y$ implies $XA \rightarrow Y$, $B^+ \subseteq (B \setminus \{XA \rightarrow Y\}) \cup \{X \rightarrow Y\})^+$ is always satisfied
  - We still need to test if $X \rightarrow Y \in B^+$
  - Which is equivalent to testing if Y is in $X^{+(B)}$

- We can remove $X \rightarrow A$ from B if:
  - $X \rightarrow A$ is in $(B \setminus \{X \rightarrow A\})^+$
  - Which is equivalent to: A is in $X^{+(B \setminus \{X \rightarrow A\})}$

Example: Dropping Attributes / FDs

F = { ABE→C, CD→E, A→E, DE→A }

- Can we drop A from ABE→C?
  - BE→C is a stronger rule
  - Hence, only if we do not add power; does F already imply BE→C ?
  - compute BE^{+F} = {B,E}, hence **NO**

- Can we drop ABE→C ?
  - Only if we can still derive it from F' = { CD→E, A→E, DE→A }
  - Compute $ABE^{+(F')}$ = ABE, hence **NO**

1. Computing Minimal Basis

Input: set of dependencies F

Output: minimal basis

Algorithm:

1. Split each rule $X \rightarrow A_1 \ldots A_k$ in F into $X \rightarrow A_1, \ldots, X \rightarrow A_k$
2. For each rule $X \rightarrow A$ and C in X, test if we can remove C from X
   Recall: we can remove C if A is in $(X \setminus \{C\})^{+B}$
3. For each rule $X \rightarrow A$ test if $X \rightarrow A$ can be removed
   Recall: we can remove $X \rightarrow A$ *from B if A in* $X^{+(B \setminus \{X->A\})}$

It is important to execute step 2 completely ***before*** *step 3*

Example: computing minimal basis (1/4)

F = { ABE→C, CD→E, A→E, DE→A }

Can we drop A from ABE→C?

BE→C is a stronger rule

only if we do not add power; hence F already implies BE→C

compute BE^{+F} = {B,E} ➔ **NO**

Example: computing minimal basis (2/4)

F = { ABE→C, CD→E, A→E, DE→A }

Can we drop B from ABE→C?

AE→C is a stronger rule

only if we do not add power; hence F already implies AE→C

compute AE^{+F} = {A,E} ➔ **NO**

Example: computing minimal basis (3/4)

F = { ABE→C, CD→E, A→E, DE→A }

Can we drop E from ABE→C?

AB→C is a stronger rule

only if we do not add power; hence F already implies AB→C

compute AB^{+F} = {A,B,E,C} ➔ **YES**

Example: computing minimal basis (4/4)

F = { AB→C, CD→E, A→E, DE→A }

Can we drop C from CD→E? ➔ **NO**

Can we drop D from CD→E? ➔ **NO**

Can we drop D from DE→A? ➔ **NO**

Can we drop E from DE→A? ➔ **NO**

Can we drop AB→C? Only if F \ { AB→C } already implies AB→C ➔ **NO**

Can we drop any of the other rules ➔ **NO**

F is now minimal; all rules are needed and cannot be made smaller.

Algorithm for Decomposing in 3NF

- In the decomposition we want to preserve every dependency
  - First minimize our set of dependencies
    - Split up all right-hand sides: A→BC is equivalent to A→B and A→C
    - Remove superfluous attributes

      AB→C, A→B is equivalent to A→C, A→B
    - Remove superfluous dependencies

      A→B and B→C imply A→C, so if we preserve A→B and B→C then also A→C is preserved
  - Make sure at least one relation supports every remaining dependency
    - Combine all FDs with the same LHS
    - Add a relation to the decomposition with attributes XY for every dependency X→Y
    - Remove superfluous relations
  - Make sure the decomposition is lossless
    - If not, add one relation where the attributes form a candidate key

2. Make sure every FD is preserved

F = { AB→C, CD→E, A→E, DE→A } is minimal

- Add following relations:
  - R_1(ABC)
  - R_2(CDE)
  - R_3(AE)
  - R_4(ADE)

2. Make sure every FD is preserved

F = { AB→C, CD→E, A→E, DE→A } is minimal

- Add following relations:
  - R_1(ABC)
  - R_2(CDE)
  - ~~R_3(AE)~~
  - R_4(ADE)
- We could drop R_3 as it is included in R_4

Algorithm for Decomposing in 3NF

- In the decomposition we want to preserve every dependency
  - First minimize our set of dependencies
    - Split up all right-hand sides: A→BC is equivalent to A→B and A→C
    - Remove superfluous attributes

      AB→C, A→B is equivalent to A→C, A→B
    - Remove superfluous dependencies

      A→B and B→C imply A→C, so if we preserve A→B and B→C then also A→C is preserved
  - Make sure at least one relation supports every remaining dependency
  - Make sure the decomposition is lossless
    - If not, add one relation where the attributes form a candidate key

3. Make sure the decomposition is lossless

F = { AB→C, CD→E, A→E, DE→A } is minimal

The decomposition already contains:

- R_1(ABC)
- R_2(CDE)
- R_4(ADE)

Is it lossless? *Use the chase*

A	B	C	D	E
a	b	c	d_1	e_1
a_1	b_1	c	d	e
a	b_2	c_1	d	e

3. Make sure the decomposition is lossless

F = { AB→C, CD→E, A→E, DE→A } is minimal

The decomposition already contains:

- R_1(ABC)
- R_2(CDE)
- R_4(ADE)

Is it lossless? *Use the chase*

A	B	C	D	E
a	b	c	d_1	e_1
a	b_1	c	d	e
a	b_2	c_1	d	e

3. Make sure the decomposition is lossless

F = { AB→C, CD→E, A→E, DE→A } is minimal

The decomposition already contains:

- R_1(ABC)
- R_2(CDE)
- R_4(ADE)

Is it lossless? *Use the chase*

A	B	C	D	E
a	b	c	d_1	e
a	b_1	c	d	e
a	b_2	c_1	d	e

3. Make sure the decomposition is lossless

F = { AB→C, CD→E, A→E, DE→A } is minimal

The decomposition already contains:

- R_1(ABC)
- R_2(CDE)
- R_4(ADE)

Is it lossless? *Use the chase* → **NO**

A	B	C	D	E
a	b	c	d_1	e
a	b_1	c	d	e
a	b_2	c_1	d	e

3. Make sure the decomposition is lossless

F = { AB→C, CD→E, A→E, DE→A } is minimal

The decomposition already contains:

- R_1(ABC)
- R_2(CDE)
- R_4(ADE)

Is it lossless? *Use the chase* → **NO**

- Add a relation with a candidate key
  - E.g.: R_5(ABD)

  - Lossless decomposition in 3NF: R_1(ABC), R_2(CDE), R_4(ADE), R_5(ABD)

Example 2: 3NF

R(Course,Teacher,Hour,Room,Student,Grade)
F = {C→TR, HR→C, HT→R, CS→G, HS→R}

1. Minimize F

Example 2: 3NF

R(Course,Teacher,Hour,Room,Student,Grade)
F = {C→T, C→R, HR→C, HT→R, CS→G, HS→R}

1. Minimize F:
    1. Can we remove H from HR→C ? $R^{+(F)}$ = R → NO
    2. Can we remove R from HR→C ? $H^{+(F)}$ = H → NO
    3. Can we remove H from HT→R ? $T^{+(F)}$ = T → NO
    4. Can we remove T from HT→R ? $H^{+(F)}$ = H → NO
    5. Can we remove C from CS→G ? $S^{+(F)}$ = S → NO
    6. Can we remove S from CS→G ? $C^{+(F)}$ = CTR → NO
    7. Can we remove H from HS→R ? $S^{+(F)}$ = S → NO
    8. Can we remove S from HS→R ? $H^{+(F)}$ = H → NO

Example 2: 3NF

R(Course,Teacher,Hour,Room,Student,Grade)
F = {C→T, C→R, HR→C, HT→R, CS→G, HS→R}

1. Minimize F:
    1. Can we remove C→T ? No
    2. C→R ? No
    3. HR→C ? No
    4. HT→R ? No
    5. CS→G ? No
    6. HS→R ? No

Example 2: 3NF

R(Course,Teacher,Hour,Room,Student,Grade)
F = {C→T, C→R, HR→C, HT→R, CS→G, HS→R}

1. Minimize F
2. Make a relation for each FD:
    1. R_1(C,R,T)
    2. R_2(C,H,R)
    3. R_3(H,R,T)
    4. R_4(C,G,S)
    5. R_5(H,R,S)

Example 2: 3NF

R(Course,Teacher,Hour,Room,Student,Grade)
F = {C→T, C→R, HR→C, HT→R, CS→G, HS→R}

1. Minimize F
2. Make a relation for each FD:

   $R_1(C,R,T)$, $R_2(C,H,R)$, $R_3(H,R,T)$, $R_4(C,G,S)$, $R_5(H,R,S)$
3. Test if lossless → Chase

C	T	H	R	S	G
c	t		r		
c		h	r		
	t	h	r		
c				s	g
		h	r	s	

Example 2: 3NF

R(Course,Teacher,Hour,Room,Student,Grade)
F = {C→T, C→R, HR→C, HT→R, CS→G, HS→R}

1. Minimize F
2. Make a relation for each FD:

   $R_1(C,R,T)$, $R_2(C,H,R)$, $R_3(H,R,T)$, $R_4(C,G,S)$, $R_5(H,R,S)$
3. Test if lossless → Chase

C	T	H	R	S	G
c	t		r		
c	t	h	r		
	t	h	r		
c	t		r	s	g
		h	r	s	

Example 2: 3NF

R(Course,Teacher,Hour,Room,Student,Grade)
F = {C→T, C→R, HR→C, HT→R, CS→G, HS→R}

1. Minimize F
2. Make a relation for each FD:

   $R_1(C,R,T)$, $R_2(C,H,R)$, $R_3(H,R,T)$, $R_4(C,G,S)$, $R_5(H,R,S)$
3. Test if lossless → Chase

C	T	H	R	S	G
c	t		r		
c	t	h	r		
c	t	h	r		
c	t		r	s	g
c		h	r	s	

Example 2: 3NF

R(Course,Teacher,Hour,Room,Student,Grade)
F = {C→T, C→R, HR→C, HT→R, CS→G, HS→R}

1. Minimize F
2. Make a relation for each FD:

   $R_1(C,R,T)$, $R_2(C,H,R)$, $R_3(H,R,T)$, $R_4(C,G,S)$, $R_5(H,R,S)$
3. Test if lossless → Chase

C	T	H	R	S	G
c	t		r		
c	t	h	r		
c	t	h	r		
c	t		r	s	g
c	t	h	r	s	

Example 2: 3NF

R(Course,Teacher,Hour,Room,Student,Grade)
F = {C→T, C→R, HR→C, HT→R, CS→G, HS→R}

1. Minimize F
2. Make a relation for each FD:

   $R_1(C,R,T)$, $R_2(C,H,R)$, $R_3(H,R,T)$, $R_4(C,G,S)$, $R_5(H,R,S)$
3. Test if lossless → Chase

   YES!

C	T	H	R	S	G
c	t		r		
c	t	h	r		
c	t	h	r		
c	t		r	s	g
c	t	h	r	s	g

Example 2: 3NF

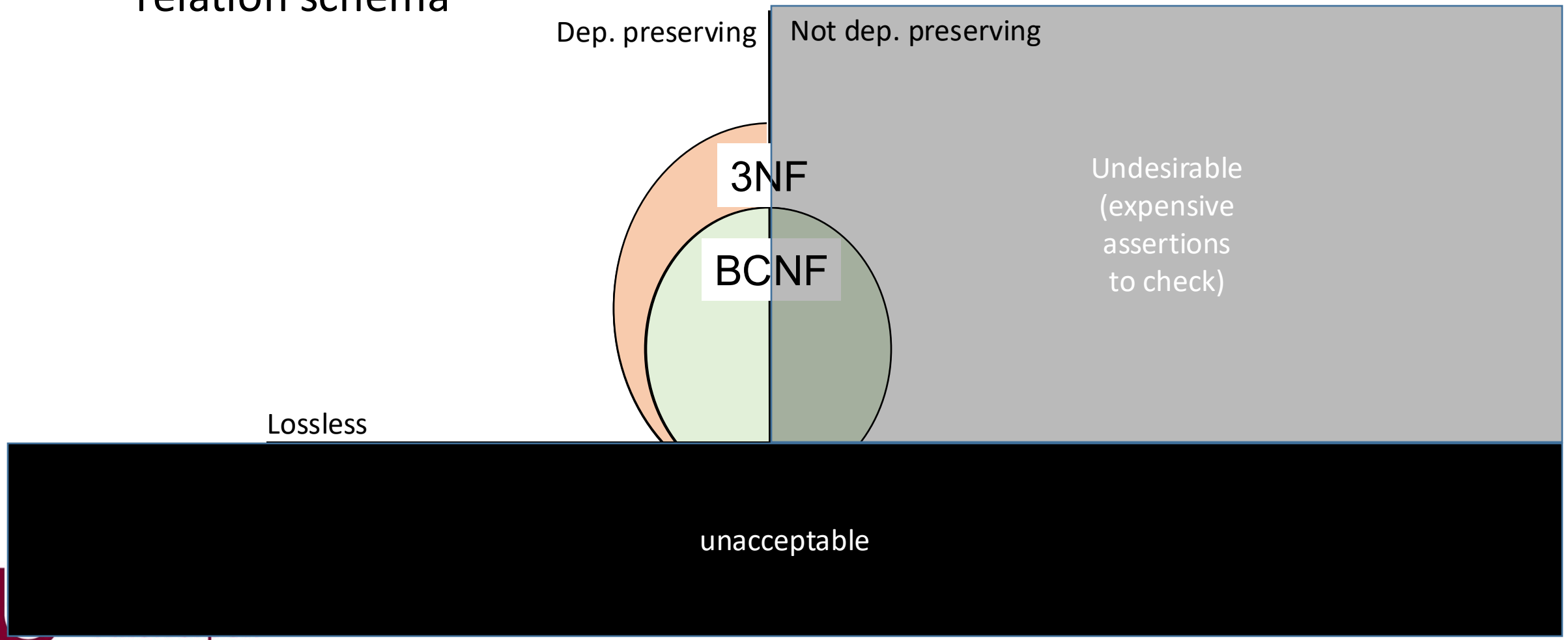
R(Course,Teacher,Hour,Room,Student,Grade)
F = {C→T, C→R, HR→C, HT→R, CS→G, HS→R}

1. Minimize F
2. Make a relation for each FD:

   $R_1(C,R,T)$, $R_2(C,H,R)$, $R_3(H,R,T)$, $R_4(C,G,S)$, $R_5(H,R,S)$
3. Test if lossless
   → Actually easier way to see here: **HS is a key and is included in R_5**

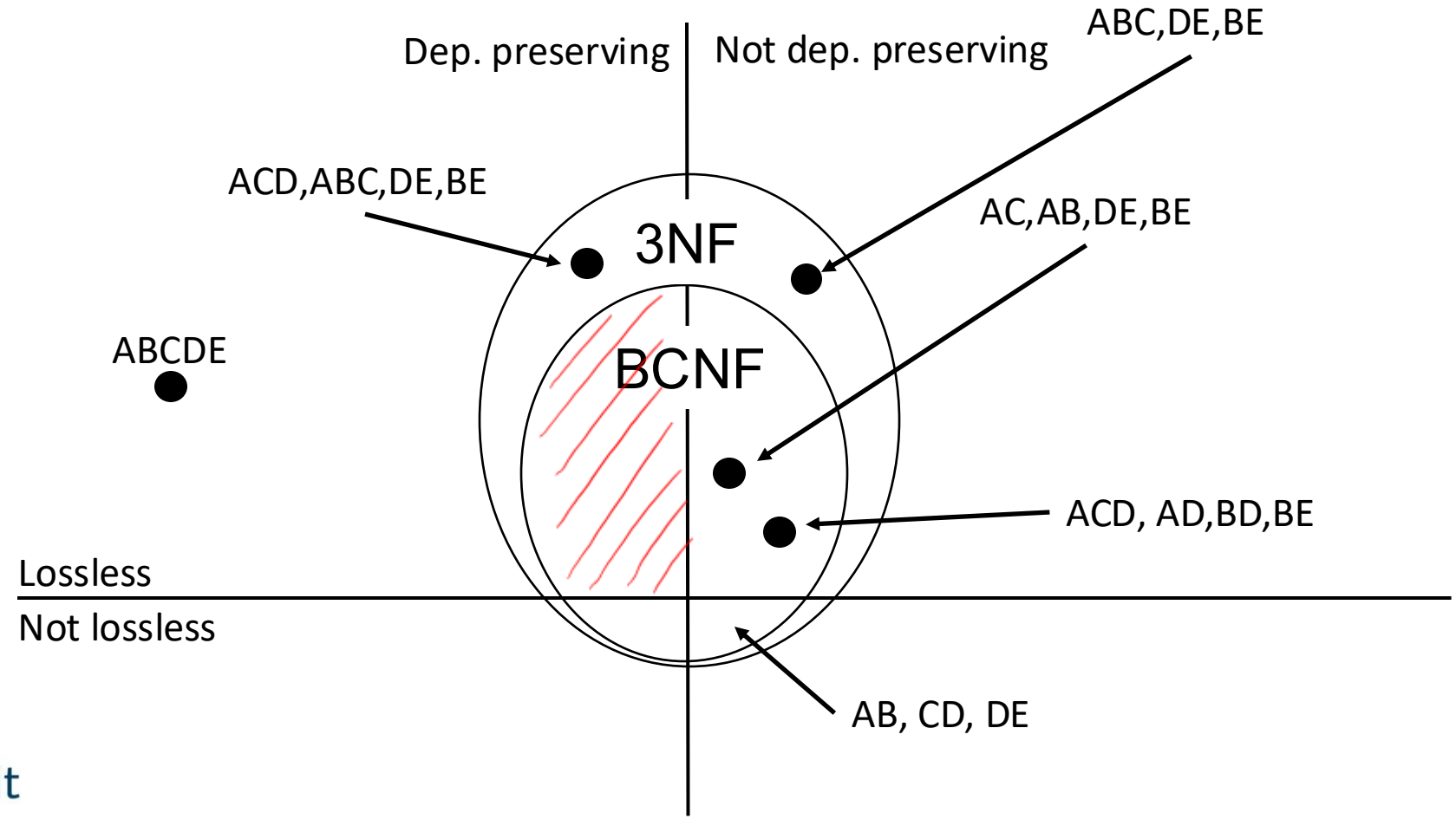
Normal forms and decompositions

- The following diagram represents all decompositions of a given relation schema



Example: Normal Forms and Decompositions

- R(ABCDE) and FD={ A→ CD, B→E, BC→A, E→D }



Summary

- Theory behind relational database design
- Functional dependencies as a tool to express potential redundancies
- Normal forms express desirable properties
  - Boyce-Codd Normal Form
  - 3rd Normal Form
- Decomposing relations to achieve normal form
  - Lossless
  - Dependency preserving